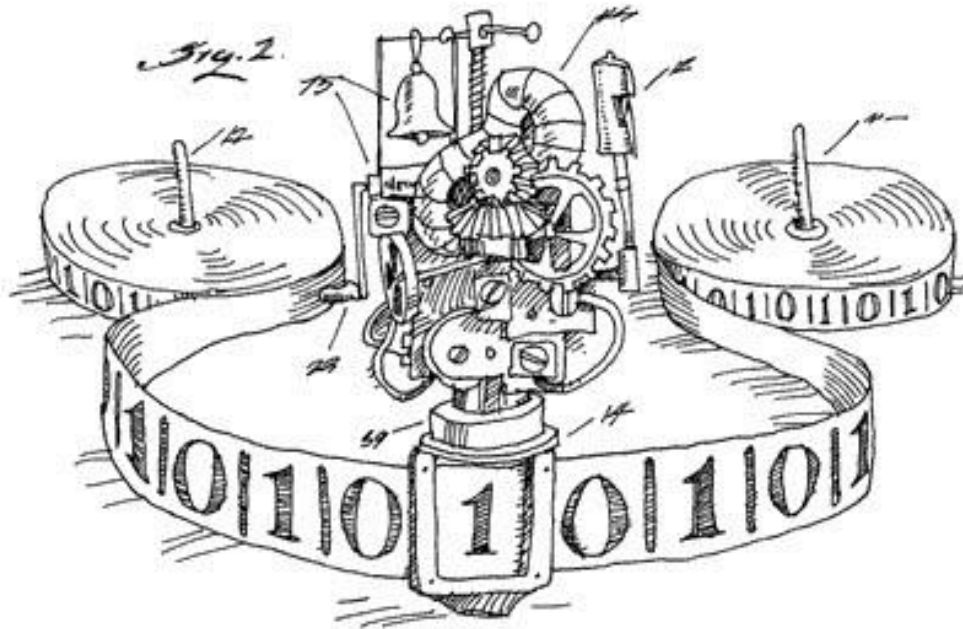
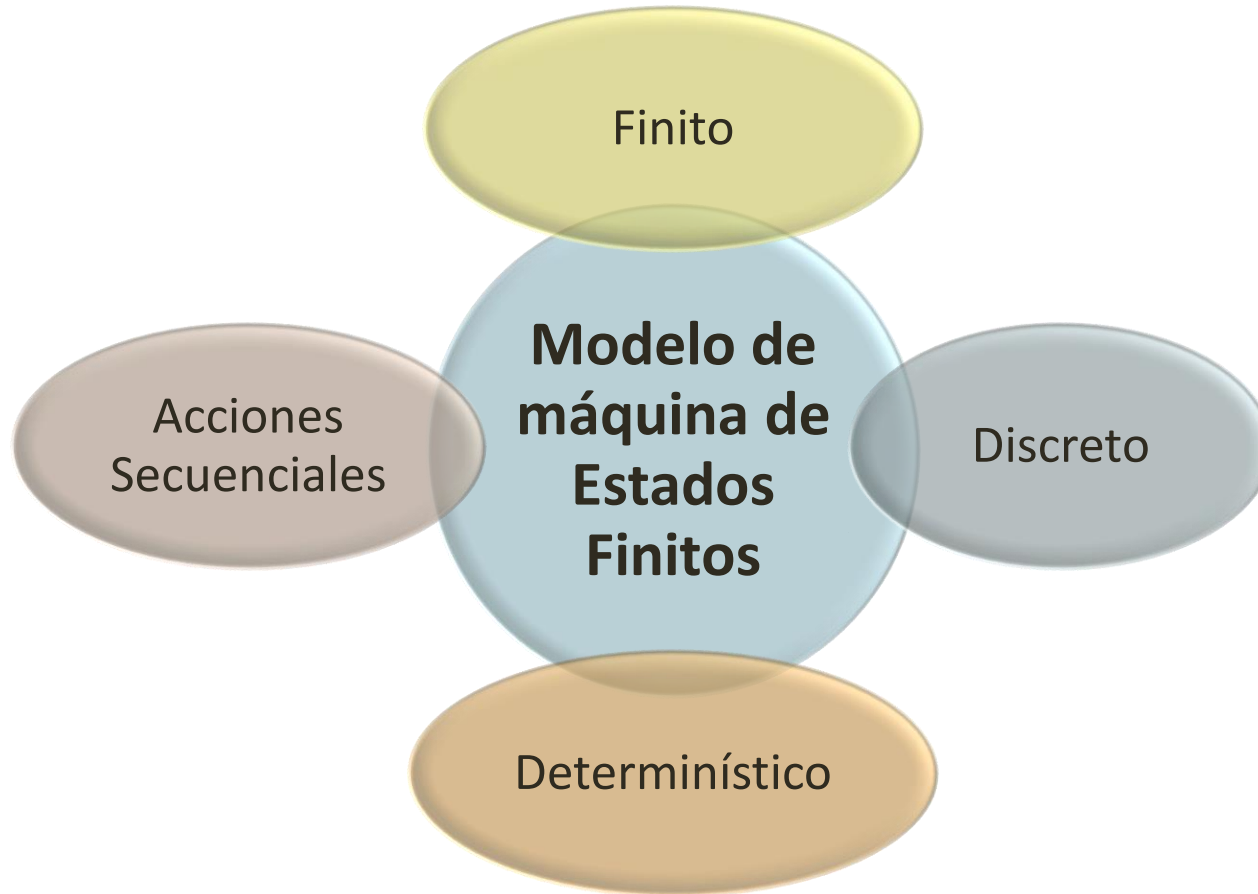




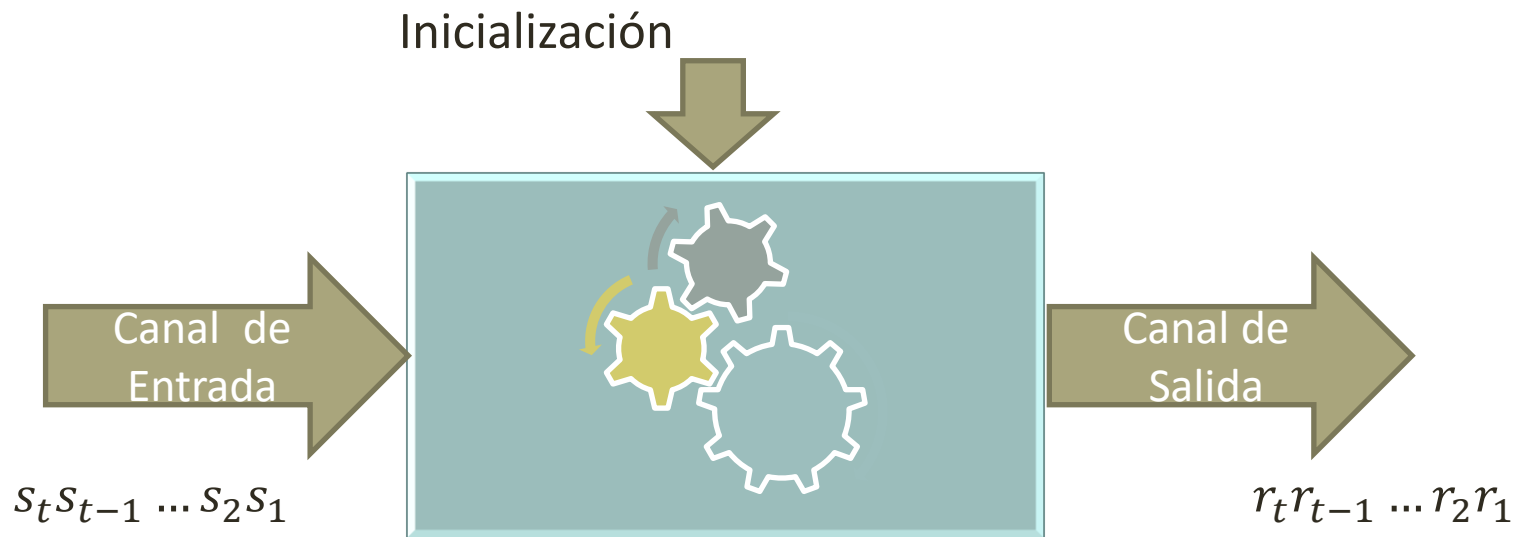
Teoría de la Computación

Autómatas de Estados Finitos

Autómatas de Estados Finitos



Autómatas de Estados Finitos



Definición M_t (Mealy)

Una máquina de estados finitos con salida asociada a transición o autómata de Mealy es una tupla de seis elementos:

$$M_t = (Q, \Sigma, \Gamma, f, g, q_0)$$

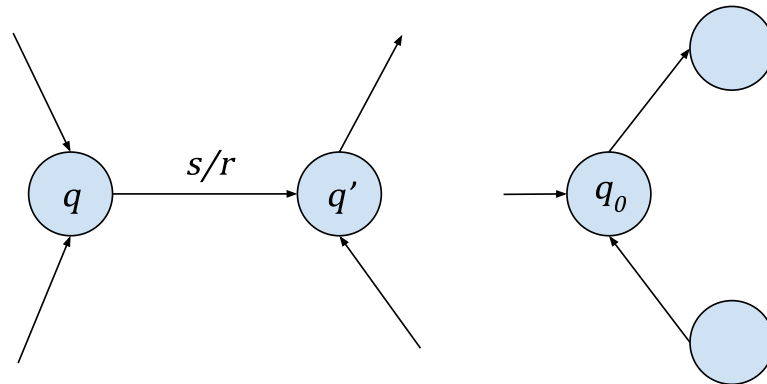
donde

- Q Conjunto finito de estados internos
- Σ Alfabeto finito de entrada (eventos de entrada)
- Γ Alfabeto finito de salida
- $f: Q \times \Sigma \rightarrow Q$ es la función de transición de estados
- $g: Q \times \Sigma \rightarrow \Gamma$ es la función de salida
- $q_0 \in Q$ es el estado inicial

Tabla y diagrama de M_t

Para representar las funciones de transición y función de salida, podemos realizar lo siguiente: $q \xrightarrow{s/r} q'$

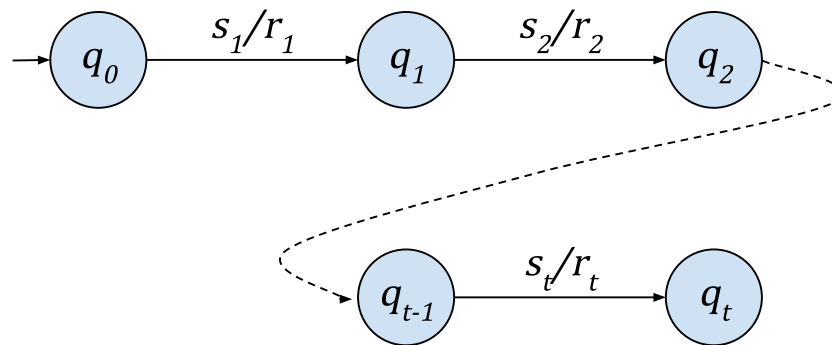
		s	
		$\dots$	
q	$\dots$	q', r	$\dots$
		$\dots$	



$$q \xrightarrow{s/r} q' \quad \text{para indicar} \quad \begin{cases} f(q, s) = q' \\ g(q, s) = r \end{cases}$$

Secuencia de estados en M_t

- Cada camino dirigido a través del diagrama de estados muestra el comportamiento de la máquina en respuesta a la secuencia de símbolos de entrada que están rotulando el camino.



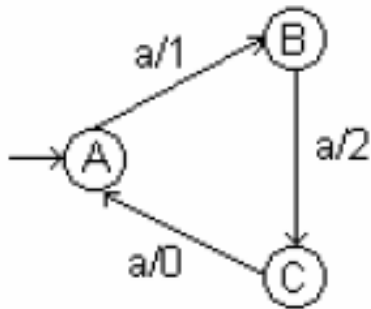
- El estímulo $s_1 s_2 \dots s_t$ hace que la máquina describa una secuencia de estados $q_0, q_1, \dots, q_t$ y genere la respuesta $r_1 r_2 \dots r_t$.

$$q_0 \xrightarrow{s_1/r_1} q_1 \xrightarrow{s_2/r_2} \dots \xrightarrow{s_t/r_t} q_t$$

Ejemplo de M_t

Diseñar una máquina de estados finitos M_1 , cuya salida sea el resultado de efectuar la operación $n \bmod m$ (para enteros $n \geq 0$ y $m > 0$, el valor de $n \bmod m$ es el del resto de la división entera n/m), donde n =longitud de un string arbitrario ($n = |\omega|$) y $m = 3$.

$$\Sigma = \{ a \}; \Gamma = \{ 0, 1, 2 \}$$



	a
A	B, 1
B	C, 2
C	A, 0

Ejemplo de M_t (x2)

- Diseñar una máquina de estados finitos M_2 que produzca una salida "1" cada vez que la paridad de la sentencia de entrada sea impar y que produzca una salida "0" cada vez que la paridad de la sentencia de entrada sea par.
- Dados los alfabetos de entrada y salida de la máquina M_3 : $\Sigma = \Gamma = \{ 0, 1 \}$. Diseñar esta máquina de forma tal que la secuencia de salida sea una réplica de la secuencia de entrada, pero demorada dos unidades.

Definición M_s (Moore)

Una máquina de estados finitos con salida asociada a estado o autómata de Moore es una tupla de seis elementos:

$$M_s = (Q, \Sigma, \Gamma, f, h, q_0)$$

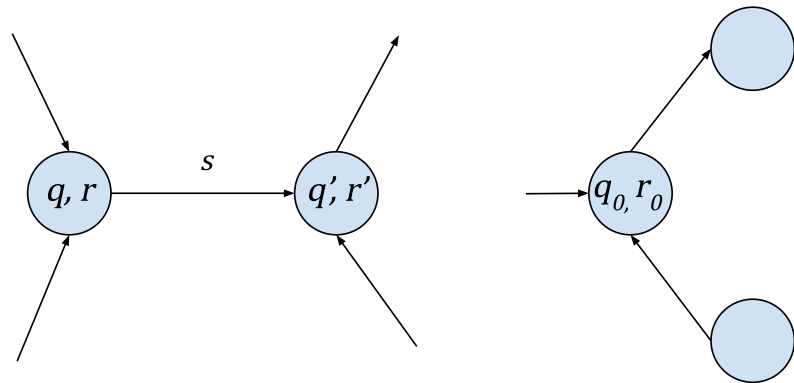
donde

- Q Conjunto finito de estados internos
- Σ Alfabeto finito de entrada
- Γ Alfabeto finito de salida
- $f: Q \times \Sigma \rightarrow Q$ es la función de transición de estados
- $h: Q \rightarrow \Gamma$ es la función de salida
- $q_0 \in Q$ es el estado inicial

Tabla y diagrama de M_s

Para representar las funciones de transición y función de salida, podemos realizar lo siguiente: $(q, r) \xrightarrow{s} (q', r')$

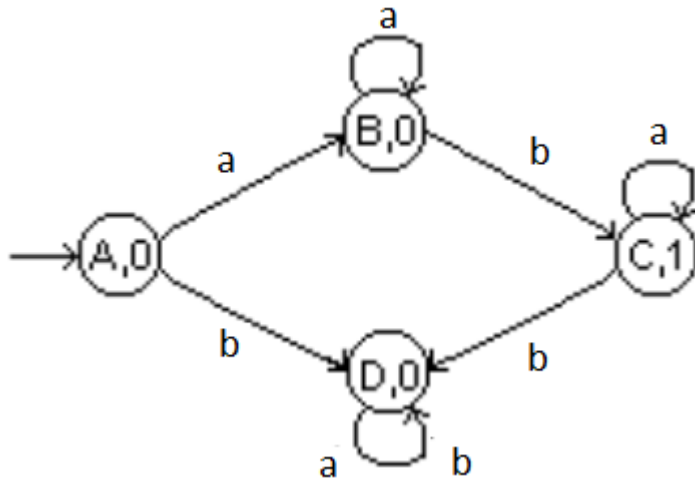
		s		
		...		
q	...	q'	...	r
		...		
q'	...		...	r'
		...		



$$(q, r) \xrightarrow{s} (q', r') \quad \text{para indicar} \quad \begin{cases} f(q, s) = q' \\ h(q) = r \\ h(q') = r' \end{cases}$$

Ejemplo de M_s

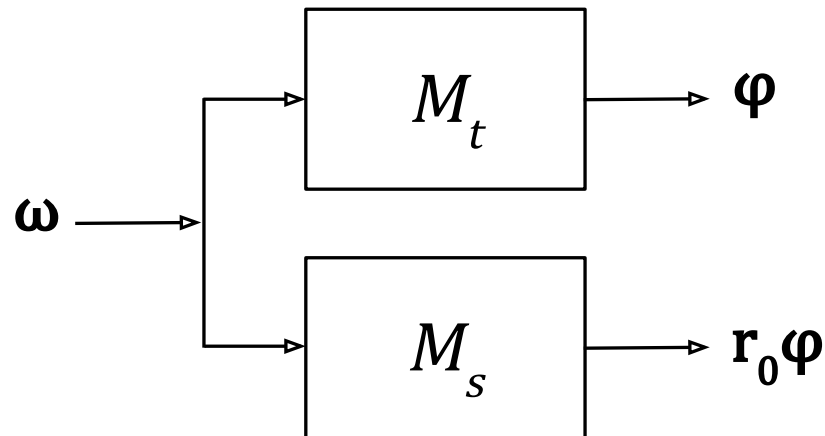
- Diseñar una máquina M_4 de modo que, dado cualquier string $\omega \in \{a, b\}^*$ (compuesto por símbolos 'a' y 'b'), produzca una salida 1 sí y sólo sí este comienza con una 'a' y contiene exactamente un sólo símbolo 'b'. En otro caso debe producir un 0.



	a	b	
A	B	D	0
B	B	C	0
C	C	D	1
D	D	D	0

Conversión de máquinas de Estados Finitos

- Una máquina con salida asociada a transición M_t y una máquina con salida asociada a estado M_s son similares si para cada estímulo posible, la respuesta de M_s es exactamente igual a la de M_t pero precedida de un símbolo arbitrario, fijo.

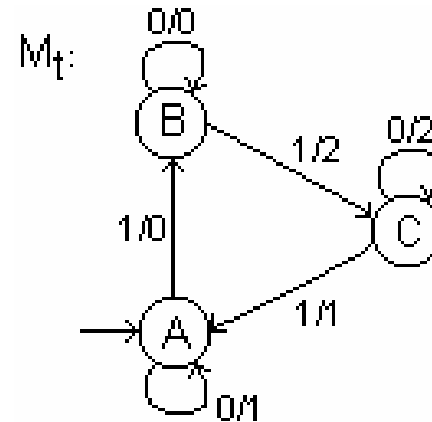
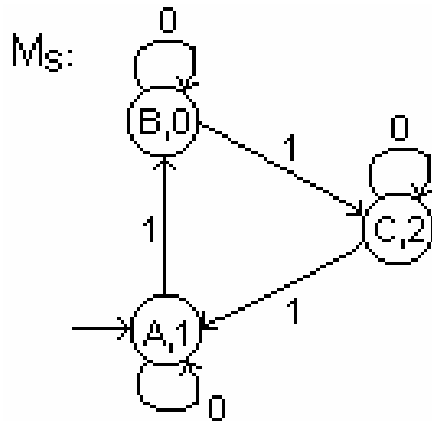


$$M_s \longrightarrow M_t$$

Partimos de: $M_s = (Q, \Sigma, \Gamma, f, h, q_0)$

Obtenemos: $M_t = (Q, \Sigma, \Gamma, f, g, q_0)$

Donde se verifica que $g(q, s) = h(f(q, s))$ para todo $q \in Q$ y $s \in \Sigma$.



$$M_t \longrightarrow M_s$$

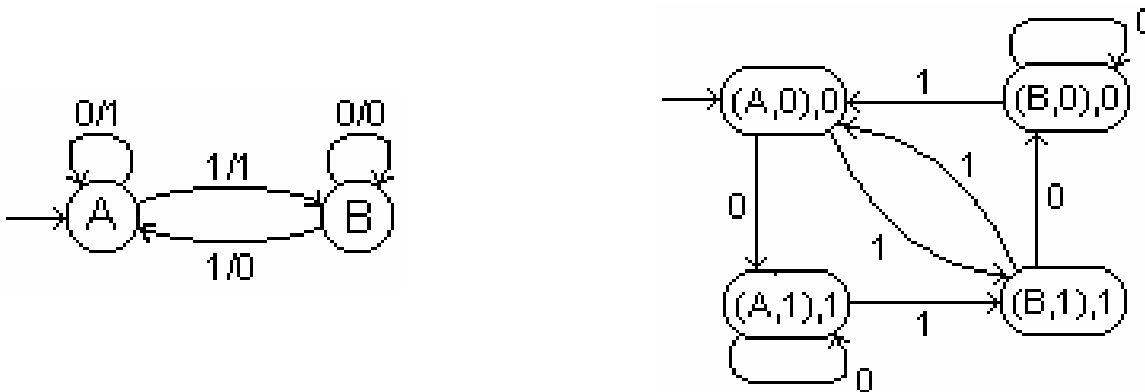
Partimos de: $M_t = (Q_t, \Sigma, \Gamma, f_t, g, q_0)$

Obtenemos: $M_s = (Q_s, \Sigma, \Gamma, f_s, h, (q_0, r_0))$

Donde: $Q_s = Q_t \times \Gamma$

$q \xrightarrow{s/r} q'$ en M_t , corresponde $((q, r'), r') \xrightarrow{s} ((q', r), r)$ en M_s para todo $r' \in \Gamma$.

$q_I \xrightarrow{s/r} q'$ en M_t , corresponde $((q_I, r_0), r_0) \xrightarrow{s} ((q', r), r)$ en M_s para un r_0 elegido arbitrariamente.



REITERAMOS

Crónica

16:21
26:5

**¡REITERAMOS! ¡TENEMOS 5
MINUTOS DE RECREO MENTAL!**

PLACASROJAS.TV



Aceptores de Estados Finitos Determinísticos

Un aceptor de estados finitos (AEFd) es una tupla de cinco elementos definida como:

$$M_d = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q es el conjunto finito de estados
- Σ es el alfabeto finito de entrada
- $\delta = Q \times \Sigma \rightarrow Q$ es la función de transición de M
- $q_0 \in Q$ es el estado inicial
- $F \subseteq Q$ son los estados finales

$q \xrightarrow{s} q'$ es una transición de M .

Aceptores de Estados Finitos

No determinísticos

Un aceptor de estados finitos (AEFnd) es una tupla de cinco elementos definida como:

$$M_{nd} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q es el conjunto finito de estados
- Σ es el alfabeto finito de entrada
- $\delta = Q \times \Sigma \rightarrow \wp(Q)$ es la función de transición de M
- $q_0 \in Q$ es el estado inicial
- $F \subseteq Q$ son los estados finales

En este caso, se toma $\wp(Q)$ como imagen de la función de transición debido a que una transición puede estar dirigida a cualquier subconjunto de estados de Q .

Aceptación de un string

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF. Se dice que M acepta un string $\omega \in \Sigma^*$ sí y sólo sí hay una secuencia de estados admisibles:

$$q_0 \xRightarrow{\omega} q'$$

para algún $q' \in F$.

Lenguaje Reconocido por M

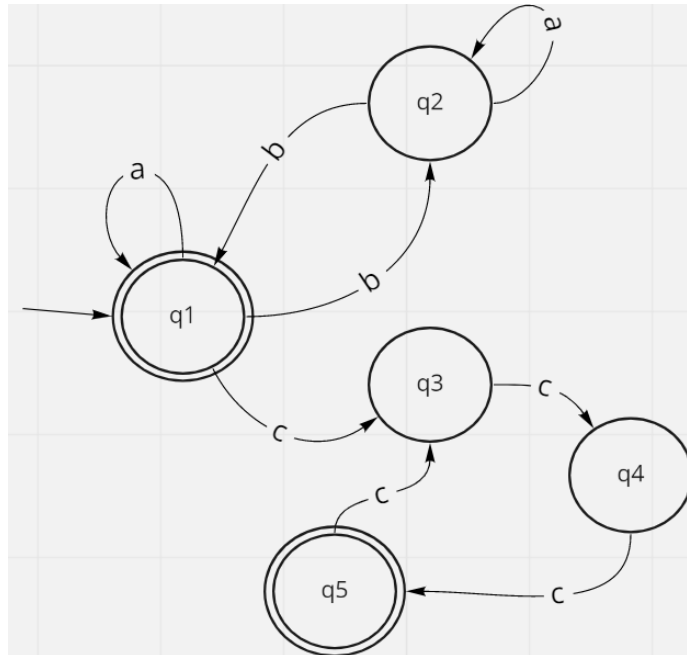
El lenguaje reconocido por M es el conjunto:

$$L(M) = \left\{ \omega \in \Sigma^* / q_0 \xRightarrow{\omega} q'; q' \in F \right\}$$

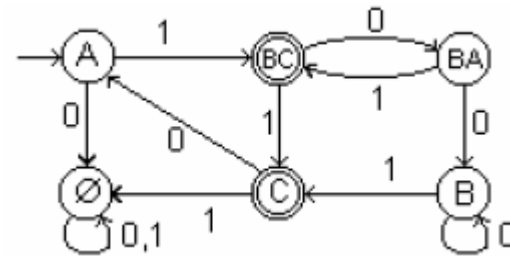
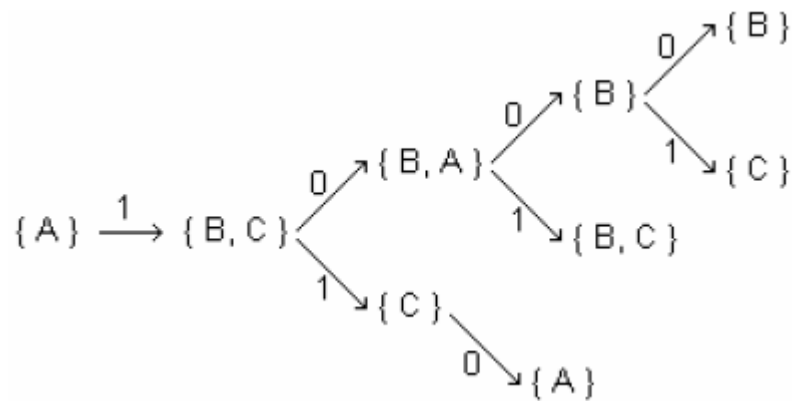
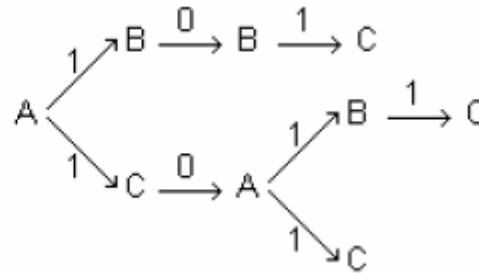
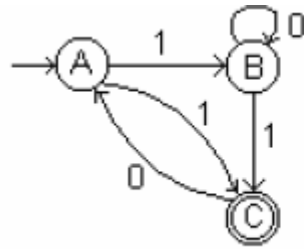
Ejemplo

- Construir un AEF que acepte el siguiente lenguaje:

$$A = \{\alpha c^{3m} / \alpha \in \{a, b\}^* \text{ y la cantidad de } b\text{'s es par y } m \geq 0\}$$



Conversión a AEF determinísticos



Cuáles serán los Estados iniciales?

Cuales serán los Estados finales?

Manos a la obra

- Supongamos que queremos construir un aceptor determinístico determine si en cualquier posición del string aparece el número 7 usando el código Morse.

The Morse Code	
A	● —
B	— ● ● ●
C	— ● — ●
D	— ● ●
E	●
F	● ● — ●
G	— — ●
H	● ● ● ●
I	● ●
J	● — — —
K	— ● —
L	● — ● ●
M	— —
N	— ●
O	— — —
P	● — — ●
Q	— — ● —
R	● — ●
S	● ● ●
T	—
U	● ● —
V	● ● ● —
W	● — —
X	— ● ● —
Y	— ● — —
Z	— — ● ●
0	— — — — —
1	● — — — —
2	● ● — — —
3	● ● ● — —
4	● ● ● ● —
5	● ● ● ● ●
6	— ● ● ● ●
7	— — ● ● ●
8	— — — ● ●
9	— — — — ●

Minimización de AEF

Preliminares

Dos estados son distinguibles si hay por lo menos un string $\omega \in \Sigma^*$, que es aceptado con el AEF inicializado en q_a y es rechazado con el AEF inicializado en q_b .

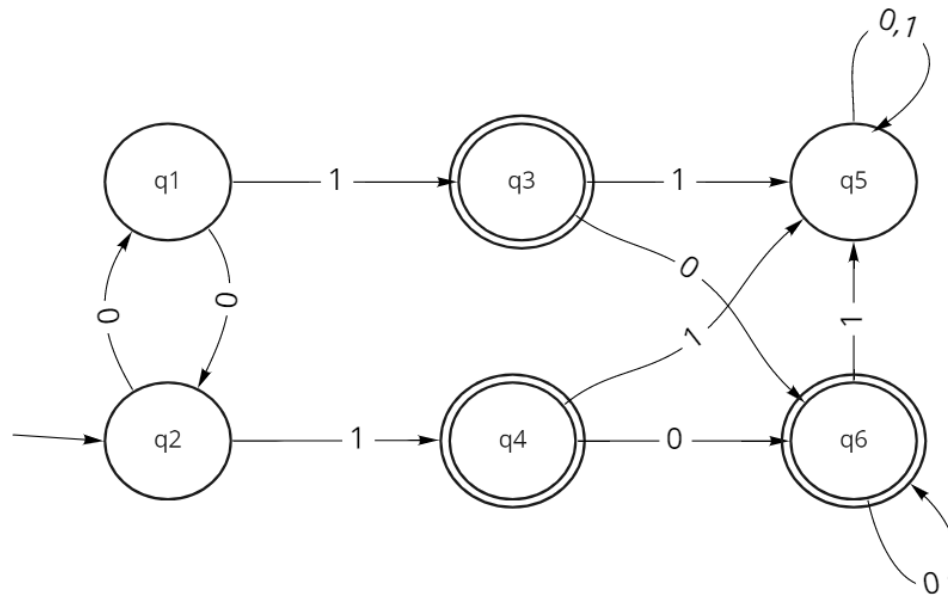
Si q_a y q_b son dos estados en un AEF, podemos combinarlos en un mismo conjunto $\{q_a, q_b\}$ si ellos no son distinguibles (equivalentes).

Finalmente, un AEF es mínimo si y solo si todos sus estados son distinguibles.

Algoritmo de Particionado

- **Etapa 1:** Generar una partición inicial P_0 de Q agrupando aquellos estados de aceptación y de no aceptación. Los estados q y q' están en el mismo bloque de P_0 si son 0-equivalentes. Tomar un contador $k = 0$.
- **Etapa 2:** Obtener P_{k+1} a partir de P_k como sigue: Los estados q y q' están en el mismo bloque de P_{k+1} sí y sólo sí:
 1. Están en el mismo bloque de P_k .
 2. Para cada $s \in \Sigma$, los estados s -sucesores de q y q' están en el mismo bloque de P_k .
- **Etapa 3:** Repetir la etapa 2 hasta que $P_{m+1} = P_m$ para algún m , en otro caso pasar a la etapa 4.
- **Etapa 4:** Llamamos a P_m la partición final de Q . Combinar los bloques de P_m y convertirlos en nuevos estados del AEF mínimo.

Ejemplo



$P_0 = \{(q1, q2, q5), (q3, q4, q6)\}$

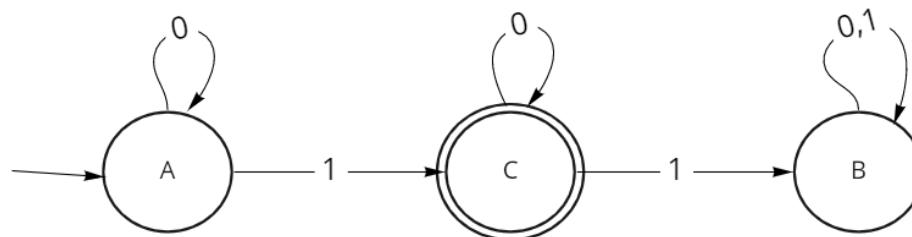
$P_1 = \{(q1, q2), (q5), (q3, q4, q6)\}$

$P_2 = \{(q1, q2), (q5), (q3, q4, q6)\}$

----A---- --B-- -----C-----

	0	1
A	A	C
B	B	B
C	C	B

Finalmente, $P_1 = P_2$.



AEF y Gramáticas Regulares

Preliminares

Dada una gramática G , indicamos como $L(G, A)$ el conjunto de strings terminales derivables en la gramática G a partir de la letra no terminal A .

$$L(G, A) = \left\{ \omega \in T^* / A \xRightarrow{*} \omega \text{ en } G \right\}$$

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF. El **conjunto final** $E(q)$ de un estado q perteneciente a M es el conjunto de strings de entrada que pueden llevar a M desde el estado q hasta un estado de aceptación.

$$E(q) = \left\{ \omega \in \Sigma^* / q \xRightarrow{\omega} q' ; q' \in F \right\}$$

Construcción de una GLD a partir de un AEF

- Dado un AEF $M = (Q, \Sigma, \delta, q_0, F)$ con conjuntos finales $\{E(q)/q \in Q\}$
- Construir una gramática lineal $G = (N, T, P, S)$ con

$$N = \{N_q/q \in Q\} \text{ tal que } L(G, N_q) = E(q), \quad q \in Q$$

Regla	Si M tiene	Luego G tiene
1	$q \xrightarrow{s} q'$	$N_q \rightarrow sN_{q'}$
2	$q \xrightarrow{s} q'; q' \in F$	$N_q \rightarrow s$
3	q_0	$S \rightarrow N_{q_0}$
4	$\{q_0\} \cap F \neq \emptyset$	$S \rightarrow \lambda$

Construcción de M a partir de una GLD

- Dada una gramática $G = (N, T, P, S)$
- Construir un AEF $M = (Q, T, \delta, q_0, \{q_f\})$ con $Q = \{q_A / A \in N\}$, con conjuntos finales $\{E(q) / q \in Q\}$ tal que $L(G, A) = E(q_A)$, $A \in N \cup \{S\}$

Regla	Si G tiene	Luego M tiene
1	$A \rightarrow sB$	$q_A \xrightarrow{s} q_B$
2	$A \rightarrow s$	$q_A \xrightarrow{s} q_f$
3	$S \rightarrow A$	q_A es inicial
4	$S \rightarrow \lambda$	q_f es inicial

Conversión entre GRLd y GRLi

- Construcción de una GRL por derecha G_i a partir de una GRL G_d

Si G_d tiene	Luego G_i tiene
$S \rightarrow sA$	$A \rightarrow s$
$A \rightarrow sB$	$B \rightarrow As$
$A \rightarrow s$	$S \rightarrow As$
$S \rightarrow s$	$S \rightarrow s$
$S \rightarrow \lambda$	$S \rightarrow \lambda$



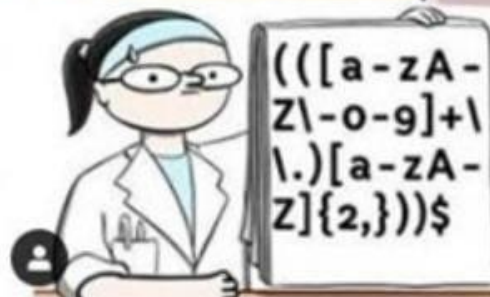
Not terrible



Not terrible



a little scary



Expresiones Regulares y AEF

Sea V un alfabeto finito. Una expresión regular en V es cualquier string finito compuesto por símbolos pertenecientes al conjunto $\{a/a \in V\} \cup \{\cup, *, (,), \lambda, \emptyset\}$

Puede ser formado de acuerdo a las siguientes reglas:

1. λ es una expresión regular.
2. $\emptyset$ es una expresión regular.
3. Si $a \in V$ entonces a es una expresión regular.
4. Si α y β son expresiones regulares entonces:
 - $(\alpha\beta)$ es una expresión regular
 - $(\alpha \cup \beta)$ es una expresión regular
 - (α^*) es una expresión regular

Equivalencia de ER

- Dos expresiones regulares son **equivalentes** sí y sólo sí ellas describen el mismo **conjunto regular**.
- Además de las más usuales correspondientes a unión y concatenación, las equivalencias más importantes se refieren a la operación de clausura (Estrella de Kleene).

- $(\alpha^*)^* = \alpha^*$
- $\alpha^* \alpha^* = \alpha^*$
- $(\lambda \cup \alpha)^* = \alpha^*$
- $\alpha \alpha^* = \alpha^* \alpha$
- $\alpha \alpha^* \cup \lambda = \alpha^*$
- $\alpha = \beta \alpha \cup \gamma \Leftrightarrow \alpha = \beta^* \gamma$

Conjunto Final

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF. El conjunto final $E(q)$ de un estado q perteneciente a M es el conjunto de strings de entrada que pueden llevar a M desde el estado q hasta un estado de aceptación.

$$E(q) = \left\{ \omega \in \Sigma^* / q \xRightarrow{\omega} q' ; q' \in F \right\}$$

Propiedades

- $\lambda \in E(q)$ sí y sólo sí $q \in F$.

Dada una transición $q \xrightarrow{s} q'$ de M para algún $q' \in Q$
Si $\omega = s\varphi$, luego $\omega \in E(q)$ sí y sólo sí $\varphi \in E(q')$.

- $L(M) = E(q_0)$

Construcción del sistema de ecuaciones de conjuntos

- Los conjuntos finales de un AEF $M = (Q, \Sigma, \delta, q_0, F)$ satisfacen un sistema de ecuaciones lineales de conjuntos (por derecha).

Por cada estado $q \in Q$:

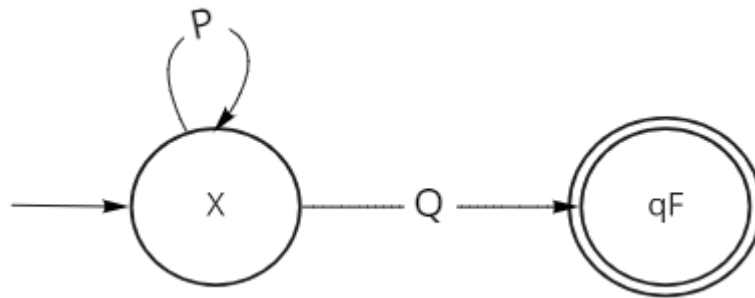
$$E(q) = \bigcup_{q' \in Q} m(q, q') E(q') \cup n(q)$$

donde

$m(q, q') = s$; tal que M tiene una transición $q \xrightarrow{s} q'$

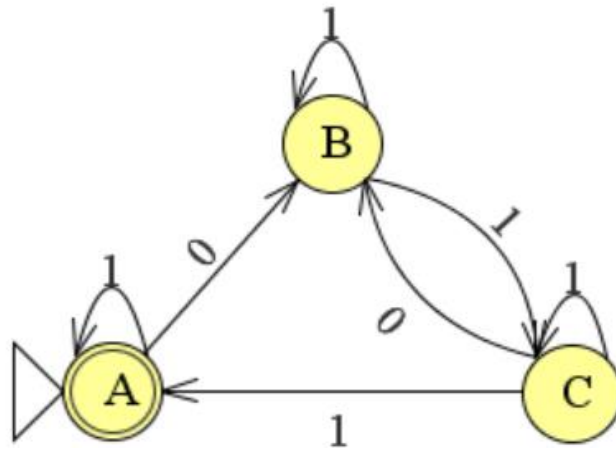
$$n(q) = \begin{cases} \lambda & \text{si } q \in F \\ \emptyset & \text{en otro caso} \end{cases}$$

Regla de Arden



$$X = P^*Q$$

Ejemplo



Las ecuaciones lineales por derecha correspondientes a este aceptor son:

- $E(A) = 1 \cdot E(A) \cup 0 \cdot E(B) \cup \lambda$
 - $E(B) = 1 \cdot E(B) \cup 1 \cdot E(C)$
 - $E(C) = 1 \cdot E(C) \cup 0 \cdot E(B) \cup 1 \cdot E(A)$
-
- $A = 1A \cup 0B \cup \lambda$
 - $B = 1B \cup 1C$
 - $C = 1C \cup 0B \cup 1A$

Obteniéndose finalmente la siguiente expresión regular:

$$(1 \cup 0(1 \cup 11^*0)^*11^*1)^*$$

Ejemplo 2

Dada la siguiente gramática G

$S \rightarrow N$	$Q \rightarrow 0T$	$X \rightarrow 1$
$N \rightarrow 0T$	$Q \rightarrow 1X$	$T \rightarrow 0$
$N \rightarrow 1X$	$Q \rightarrow 0Q$	
$N \rightarrow 0Q$	$Q \rightarrow 1Q$	
$N \rightarrow 1Q$		

- Construir un AEF que acepte el lenguaje generado por esta gramática.
- Cuáles son las características del lenguaje $L(G)$ generado a partir de esta gramática?

Expresiones Regulares

- <https://regexr.com/>

Exp	Character Classes
<code>[A-Z]</code>	Match any character in the set.
<code>[^ABC]</code>	Match any character that is not in the set.
<code>[A-Z]</code>	Matches a character having a character code between the two specified characters inclusive.
<code>.</code>	Matches any character except linebreaks. Equivalent to <code>[\n\r]</code> .
<code>\w</code>	Matches any word character (alphanumeric & underscore). Only matches low-ascii characters (no accented or non-roman characters). Equivalent to <code>[A-Za-z0-9_]</code>
<code>\W</code>	Matches any character that is not a word character (alphanumeric & underscore). Equivalent to <code>^[A-Za-z0-9_]</code>

- <https://www.youtube.com/watch?v=sXQxhojSdZM>

Expresiones Regulares

- <https://regexr.com/>

Exp	Character Classes
<code>\d</code>	Matches any digit character (0-9). Equivalent to <code>[0-9]</code> .
<code>\D</code>	Matches any character that is not a digit character (0-9). Equivalent to <code>[^0-9]</code> .
<code>\s</code>	Matches any whitespace character (spaces, tabs, line breaks).
<code>\S</code>	Matches any character that is not a whitespace character (spaces, tabs, line breaks).

Exp	Groups & References
<code>(ABC)</code>	Groups multiple tokens together and creates a capture group for extracting a substring or using a backreference.

Expresiones Regulares

- <https://regexr.com/>

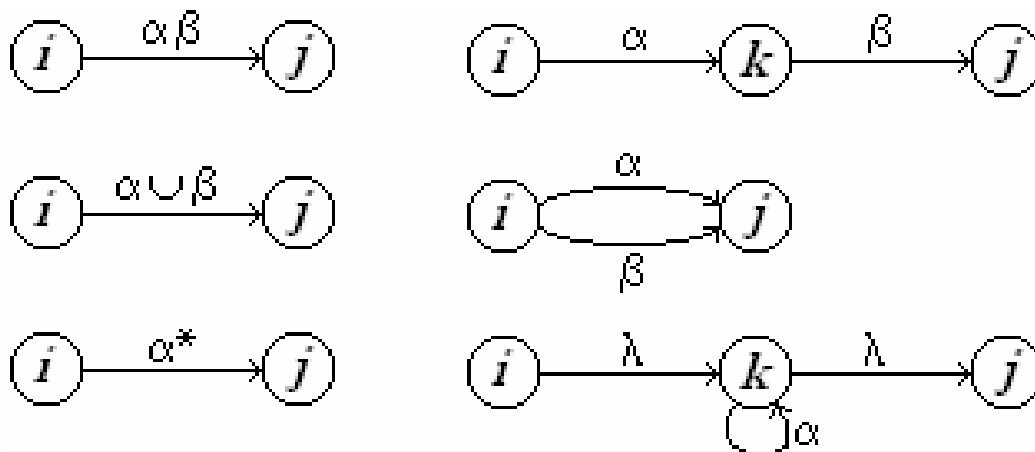
Exp	Quantifiers & Alternation
+	Matches 1 or more of the preceding token.
*	Matches 0 or more of the preceding token.
{1, 3}	Matches the specified quantity of the previous token. {1,3} will match 1 to 3. {3} will match exactly 3. {3,} will match 3 or more.
?	Matches 0 or 1 of the preceding token, effectively making it optional.
	Acts like a boolean OR. Matches the expression before or after the . It can operate within a group, or on a whole expression. The patterns will be tested in order.

Ejemplos

	Expresión
Identificar todas las palabras que comienzan con mayúsculas en un texto.	<code>([A-Z])\w+</code>
Texto entre corchetes.	<code>\[(.*?)\]</code>
Detección de una dirección de email.	<code>([\w\.]++)@([\w\.]++)\.([\w+])</code>
Detección de un comentario en Java.	<code>\/\/(?![\S]{2,}\. [\w]) .* \/*(. \n)+?*\/</code>
Texto que no contiene 'ab' ni 'cd'.	<code>^(?: (?!ab cd) .)+\$</code>

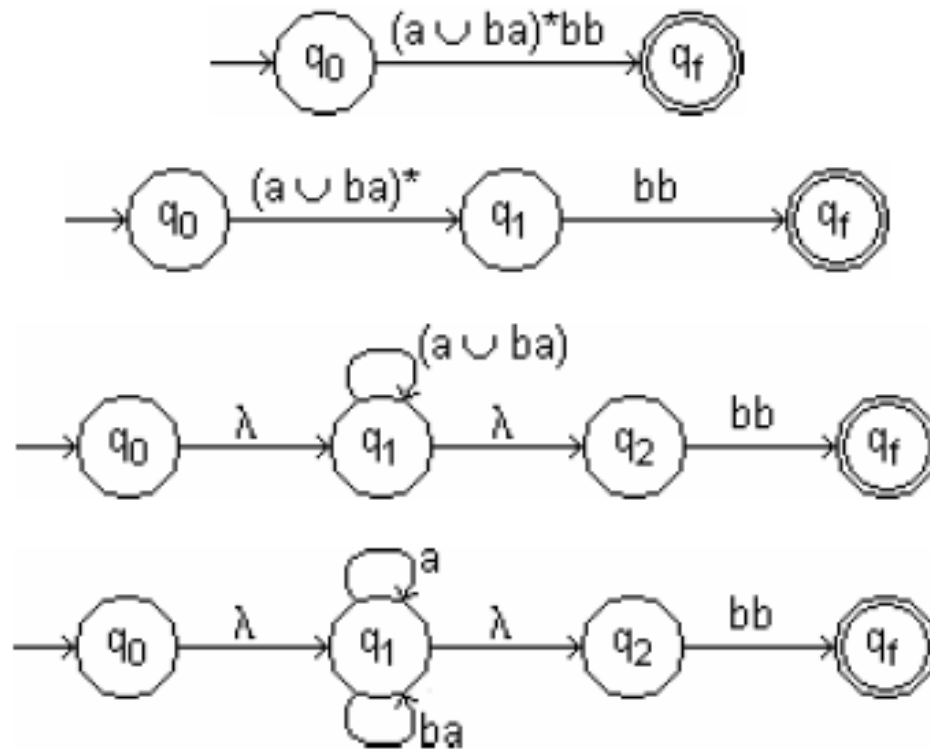
Construcción de un AEF a partir de una ER

- Se parte de una máquina M_1 con un único estado inicial, un único estado final, y una transición entre ellos rotulada por la expresión regular en cuestión; el procedimiento consiste en transformar gradualmente M_1 en M_2 , luego en M_3 , etc., hasta llegar a una M_n tal que sus transiciones estén rotuladas por símbolos del alfabeto (o bien por el string vacío).



Ejemplo

Dada la ER $\alpha = (a \cup ba)^*bb$, obtener el AEF M tal que $L(M) = \alpha$.

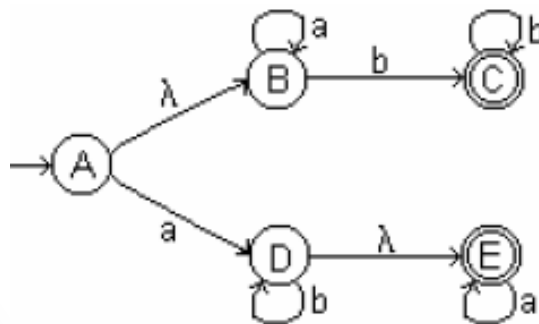


AEF con transiciones λ

Un aceptor de estados finitos con transiciones λ ($AEF - \lambda$) es un aceptor no determinístico $M = (Q, \Sigma, \delta, q_0, F)$ en el que la función de transición está definida como:

$$\delta: Q \times (\Sigma \cup \{\lambda\}) \rightarrow \wp(Q)$$

Las transiciones λ permiten al autómatata cambiar de estado sin leer ningún símbolo de la entrada.



	a	b	λ	
A	D		B	
B	B	C		
C		C		1
D		D	E	
E	E			1

λ -Clausura

Para un estado $q \in Q$, la λ -clausura de q , simbolizada $\lambda[q]$, es el conjunto de estados de M a los que se puede llegar desde q por 0, 1 o más transiciones λ .

$$\lambda[q] = \{q\} \cup \left\{ q' \in Q / q \xrightarrow{\lambda} \dots \xrightarrow{\lambda} q' \text{ en } M \right\}$$

o también:

$$\lambda[q] = \{q\} \cup \lambda[q']; \quad \text{para todo } q' \in \delta(q, \lambda)$$

λ -Clausura de un conjunto

La λ -clausura de un conjunto de estados $\{q_1, q_2, \dots, q_n\}$ se define por:

$$\lambda[\{q_1, q_2, \dots, q_n\}] = \lambda[q_1] \cup \lambda[q_2] \cup \dots \cup \lambda[q_n]$$

Eliminación de Transiciones λ

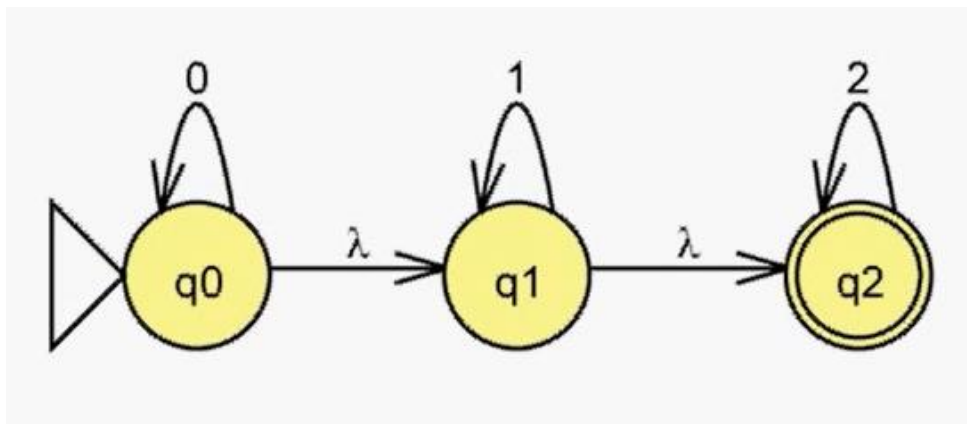
Partimos de: $M = (Q, \Sigma, \delta, q_0, F)$ un AEF $-\lambda$

Obtenemos: $M' = (Q, \Sigma, \delta', q_0, F')$ un AEF sin transiciones λ

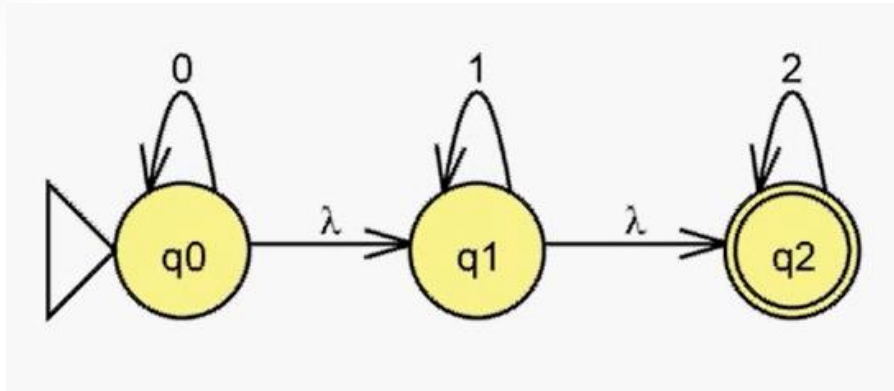
donde $\delta': Q \times \Sigma \rightarrow \wp(Q)$ tal que $\delta'(q, s) = \lambda[\delta(\lambda[q], s)]$

$$q \in Q; s \in \Sigma$$

y $F' = \{q \in Q / \lambda[q] \cap F \neq \emptyset\}$.



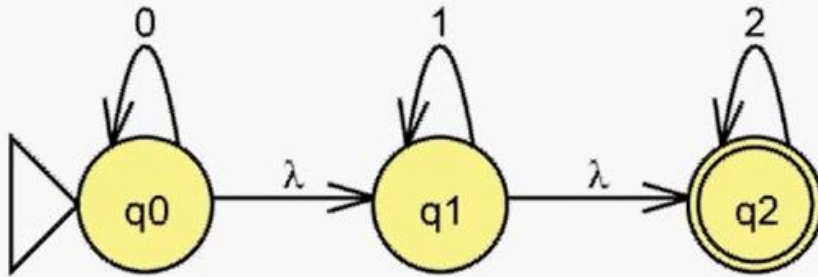
Ejemplo. Paso 1



Determinar la λ -clausura de cada estado

- $\lambda[q2] = \{q2\}$
- $\lambda[q1] = \{q1\} \cup \lambda[q2] = \{q1, q2\}$
- $\lambda[q0] = \{q0\} \cup \lambda[q1] = \{q0, q1, q2\}$

Ejemplo. Paso 2

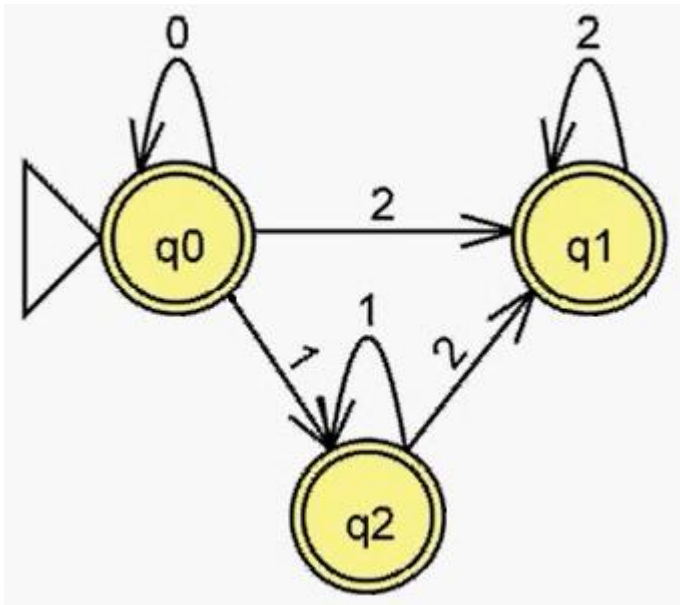


- $\lambda[q2] = \{q2\}$
- $\lambda[q1] = \{q1, q2\}$
- $\lambda[q0] = \{q0, q1, q2\}$

- $\delta'(q0, 0) = \lambda[\delta(\lambda[q0], 0)] = \lambda[\delta(\{q0, q1, q2\}, 0)] = \lambda[\{q0\}] = \{q0, q1, q2\}$
- $\delta'(q0, 1) = \lambda[\delta(\lambda[q0], 1)] = \lambda[\delta(\{q0, q1, q2\}, 1)] = \lambda[\{q1\}] = \{q1, q2\}$
- $\delta'(q0, 2) = \lambda[\delta(\lambda[q0], 2)] = \lambda[\delta(\{q0, q1, q2\}, 2)] = \lambda[\{q2\}] = \{q2\}$
- $\delta'(q1, 0) = \lambda[\delta(\lambda[q1], 0)] = \lambda[\delta(\{q1, q2\}, 0)] = \lambda[\emptyset] = \emptyset$
- $\delta'(q1, 1) = \lambda[\delta(\lambda[q1], 1)] = \lambda[\delta(\{q1, q2\}, 1)] = \lambda[\{q1\}] = \{q1, q2\}$
- $\delta'(q1, 2) = \lambda[\delta(\lambda[q1], 2)] = \lambda[\delta(\{q1, q2\}, 2)] = \lambda[\{q2\}] = \{q2\}$
- $\delta'(q2, 0) = \lambda[\delta(\lambda[q2], 0)] = \lambda[\delta(\{q2\}, 0)] = \lambda[\emptyset] = \emptyset$
- $\delta'(q2, 1) = \lambda[\delta(\lambda[q2], 1)] = \lambda[\delta(\{q2\}, 1)] = \lambda[\emptyset] = \emptyset$
- $\delta'(q2, 2) = \lambda[\delta(\lambda[q2], 2)] = \lambda[\delta(\{q2\}, 2)] = \lambda[\{q2\}] = \{q2\}$

Ejemplo

AEF sin transiciones λ



	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\{\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\{\}$	$\{\}$	$\{q_2\}$